MARRI LAXMAN REDDY
INSTITUTE OF TECHNOLOGY & MANAGEMENT
(AN AUTONOMOUS INSTITUTION)
(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)
NAAC Accredited Institution with 'A' Grade & Recognized Under Section2(f) & 12(B)of the UGC act,1956

AN INDUSTRIAL ORIENTED MINI PROJECT

On

DIGITAL DESIGN OF ARITHMETIC LOGIC UNIT (ALU)

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD

In partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

By

UDAYAGIRI AVINASH (247Y5A0426)

Under the Guidance of

Mr. G. Siva Sankar Varma, Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

April, 2026

Date:

CERTIFICATE

This is to certify that the mini project work entitled "**DIGITAL DESIGN OF 32-BIT ARITHMETIC LOGIC UNIT(ALU)**" work done by **UDAYAGIRI AVINASH (247Y5A0426)** student of Department of Electronics and Communication Engineering, is a record of Bonafide work carried out by the members during a period from **January, 2026** to **April, 2026** under the supervision of **Mr. G. Siva Sankar Varma, Assistant Professor**. This Mini Project is done as a fulfilment of obtaining Bachelor of Technology Degree to be awarded by Jawaharlal Nehru Technological University Hyderabad, Hyderabad. The matter embodied in this project report has not been submitted by us to any other university for the award of any other degree.

UDAYAGIRIAVINASH

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Date: **Mr. G. Siva Sankar Varma**

The Viva-Voce Examination of above students, has been held on……

Head of the Department Principal/Director

External Examiner

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my guide **Mr. G. Siva Sankar Varma, Assistant Professor, Electronics & Communication Engineering**, for his excellent guidance and invaluable support, which helped me accomplish the B-Tech (ECE) degree and prepared me to achieve more life goals in the future. His total support of my dissertation and countless contributions to my technical and professional development made for a truly enjoyable and fruitful experience. Special thanks are dedicated for the discussions we had on almost every working day during my project period and for reviewing my dissertation.

We are very much grateful to my Project Coordinator**, Dr. R Prabhakar, Assistant Professor, Electronics & Communication Engineering**, MLRITM, Dundigal, Hyderabad, who has not only shown utmost patience, but was fertile in suggestions, vigilant in directions of error and has been infinitely helpful.

We are extremely grateful to **Dr. N Srinivas, Head of the Department, Electronics & Communication Engineering**, MLRITM, Dundigal, Hyderabad, for the moral support and encouragement given in completing my project work.

We wish to express deepest gratitude and thanks to **Dr. P. Sridhar, Director & Dr. R. Murali Prasad, Principal,** for their constant support and encouragement in providing all the facilities in the college to do the project work.

We would also like to thank all our faculties, administrative staff and management of MLRITM, who helped me to completing the mini project.

On a more personal note, I thank my **beloved parents and friends** for their moral support during the course of our project.

TABLE OF CONTENTS

Content	Page No.
Certificate	*Ii*

Acknowledgements			*Iii*
Table of Contents			*Iv*
List of Figures			*vii*
List of Abbreviations			*viii*
Abstract			*Ix*
Chapter 1: Introduction			**10-11**
	1.1	Background & Importance	10
	1.2	Overview of 32-bit ALU	10
	1.3	Motivation	11
	1.4	Problem Statement	11
	1.3	Scope of the Project	11
Chapter 2: Literature Survey			**12**
	2.1	Implementation and Verification of 32-bit ALU	12
	2.2	Design and Implementation of Low Power 32-bit ALU	12
	2.3	Performance Analysis of 32-bit ALU	12
	2.4	Key Challenges Highlighted in Literature	12
	2.5	The Project's Contribution in Literature	12
Chapter 3: Objectives			**13**
	3.1	General Objectives	13
	3.2	Specific Objectives	13
	3.3	Outcome Statement	13
Chapter 4: Methodology			**14-15**
	4.1	Introduction	14

	4.2	Tools and Technology Used	14
	4.3	Design Flow Steps	15
Chapter 5: Design of 32-bit ALU			**16**
	5.1	Introduction	16
	5.2	Steps in Design	16
	5.3	Input Signal Description	16
	5.4	Output Signal Description	16
	5.5	Summary	16
Chapter 6: Implementation			**17-21**
	6.1	Verilog Code	17-18
	6.2	Code Explanation	19
	6.3	Test bench for 32-bit ALU	19-20
	6.4	Summary	21
Chapter 7: Results			**22-33**
	7.1	Simulation	22-23
	7.2	Synthesis	23
	7.3	Reports	24-25
	7.3.1	Area Report	24
	7.3.2	Power Report	24
	7.3.3	Timing report	25
	7.4	Physical Design	26-33
	7.4.1	Floor plan	26
	7.4.2	Power plan	27

	7.4.3	Placement and Route	28-29
	7.4.4	Clock Tree synthesis	29-31
	7.4.5	Optimization	31-32
	7.4.6	GDSII File	33

Chapter 8: Conclusion			**34-37**
	8.1	Advantages	35
	8.2	Disadvantages	35
	8.3	Real Time Applications	36
Chapter 9: Future Scope			**37**
References			38

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
Figure 1	General Block Diagram of a 32-bit ALU	10
Figure 2	32-bit ALU Architecture	2
Figure 3	Simulation Waveforms of 32-bit ALU	22
Figure 4	Schematic diagram of 32-bit ALU	23
Figure 5	Power Report of 32-bit ALU	24
Figure 6	Area Report of 32-bit ALU	24
Figure 7	Timing Report of 32-bit ALU	25
Figure 8	Floorplan of 32-bit ALU	26
Figure 9	Power plan of 32-bit ALU	27
Figure 10	Placement of 32-bit ALU	28
Figure 11	Placement and Routing of 32-bit ALU	29
Figure 12	Clock Tree Synthesis	29
Figure 13	Optimized Power Report	31
Figure 14	Optimized Timing Report for Setup	32
Figure 15	GDS File of 32-bit ALU	33

LIST OF ABBREVIATIONS

ALU	Arithmetic Logic Unit
RTL	Register Transfer Level
CTS	Clock Tree Synthesis
PD	Physical Design
GDS	Graphic Data System

WNS	Worst Negative Slack
TNS	Total Negative Slack
DRC	Design Rule Check
STA	Static Time Analysis
LVS	Layout Versus Schematic
FA	Full Adder
VLSI	Very Large-Scale Integration
GUI	Graphical User Interface
HDL	Hardware Description Language
SOC	System on Chip
PNR	Placement and Route
CADENCE	Cadence Design Systems (EDA Tool Vendor)
DFT	Design for Testability
CDC	Clock Domain Crossing
PPA	Power, Performance, Area
MUX	Multiplexer

ABSTRACT

This project focuses on the ASIC design and implementation of a 32-bit Arithmetic Logic Unit (ALU), targeting efficient computation with optimized power and performance. The design includes essential

arithmetic and logical operations such as addition, subtraction, AND, OR, XOR, XNOR, and NOT, along with buffer functionality. A ripple carry adder is used for arithmetic operations, while subtraction is implemented using the two's complement method to ensure hardware efficiency.

The ALU is described using Verilog HDL and functionally verified through a comprehensive testbench that evaluates all operations, including edge cases such as zero condition, carry generation, and overflow scenarios. Status flags such as Zero, Carry, and Sign are generated to indicate the condition of the output, enabling processor-level decision-making.

The backend ASIC design flow is carried out using Cadence tools. Synthesis is performed using Genus, while physical design including floor planning, placement, and routing is implemented using Innovus. Power analysis is conducted to evaluate leakage, internal, and switching power components, with switching power identified as the dominant factor. Design Rule Checking (DRC) is performed to ensure layout compliance with fabrication constraints, and initial violations are analyzed and reduced through optimization techniques.

Post-implementation results confirm that the design achieves functional correctness and meets basic performance requirements in terms of power and layout feasibility. The 32-bit ALU design is modular, scalable, and suitable for integration into digital processors and VLSI systems, providing a strong foundation for further enhancements in speed and power optimization.

CHAPTER-1 INTRODUCTION

1.1 Background and Importance

In modern digital systems, especially in SoC architectures, the Arithmetic Logic Unit (ALU) is a key component responsible for performing arithmetic and logical operations. Its performance and power efficiency directly affect overall system speed and reliability.

A 32-bit ALU enables efficient processing of large data and supports operations like addition, subtraction, AND, OR, XOR, and XNOR. In ASIC design, the ALU must meet constraints such as power, area, and timing while maintaining correct functionality.

An optimized ALU improves system performance and is widely used in processors, DSP systems, and embedded applications, making it an essential part of modern VLSI design.

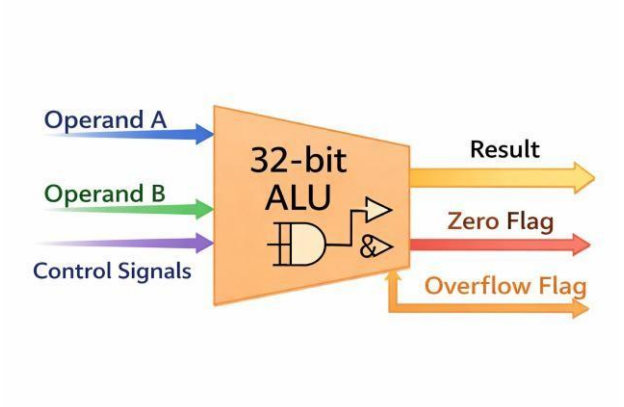

Fig 1: General block diagram of 32-bit ALU

1.2 Overview of a 32-bit ALU

A 32-bit ALU performs arithmetic and logical operations on two inputs (A and B) based on a control signal. It includes arithmetic units (adder/subtractor) and logical units (AND, OR, XOR, XNOR, NOT) that operate in parallel.

Addition is implemented using a ripple carry adder, and subtraction is done using 2's complement. A multiplexer selects the final output based on the select signal.

The ALU also generates flags such as Zero, Carry, and Sign, which indicate the result condition. Its parallel operation and modular design enable fast and efficient computation in digital systems.

1.3 Motivation

In modern digital systems, high-speed computation and efficient data processing are essential. The ALU is a core component that performs arithmetic and logical operations, directly affecting system performance.

Traditional ALU designs may suffer from delay, higher power consumption, and inefficient resource usage. Therefore, designing an optimized 32-bit ALU is important for improving speed and efficiency.

This project focuses on designing a 32-bit ALU using Verilog and implementing it through ASIC design flow, providing practical knowledge of digital design and VLSI concepts.

1.4 Problem Statement

In modern digital systems, the ALU plays a key role in performing arithmetic and logical operations, directly affecting system performance. Designing a 32-bit ALU with balanced speed, power, and area is a major challenge.

This project aims to design an ASIC-compatible 32-bit ALU that performs operations like addition, subtraction, and logical functions while ensuring correct output and flag generation. The design focuses on efficient operation selection, reduced delay, optimized power, and proper functional verification for accurate performance.

1.5 Scope of the Project

This project focuses on the schematic design, simulation, and analysis of an Asynchronous FIFO using the Cadence tools. The scope includes:

The design will include key components such as:

- Write and Read Pointers
- Dual-clock Synchronizers

the verification side, the scope includes:

- Development of a comprehensive test-bench using System Verilog
- Assertion-based verification (ABV) to validate protocol integrity
- Simulation and waveform analysis to ensure functional correctness under various corner cases and clock scenarios

Additionally, the project may explore **power and area optimization**, along with optional **DFT (Design for Testability)** enhancements for scan chain insertion.

CHAPTER 2 LITERATURE SURVEY

2.1 Implementation and Verification of 32-bit ALU

Authors: Kannan Nithin K.V. et al.**Publication:** EAI Transactions, 2024

This paper presents the design of a 32-bit ALU using CMOS technology for low power and high-speed applications. The ALU consists of arithmetic and logic units performing operations like AND, OR, XOR, XNOR, addition, and subtraction. The design uses multiplexers and full adders for efficient implementation. Simulation is performed using Cadence tools, and results show reduced power consumption with acceptable delay.

2.2 Design and Implementation of Low Power 32-bit ALU

Authors: Nazia Khan, Pankaj Soni **Publication:** 2017

This work focuses on designing a low-power 32-bit ALU using VHDL. The design is optimized for reduced area and power consumption and verified using simulation tools, though it does not include ASIC backend implementation.

2.3 Performance Analysis of 32-bit ALU

Author: Abhijit Kumar Mukhopadhyay **Publication:** 2014

This paper analyses a 32-bit ALU using different logic styles such as CMOS and transmission gate. It compares performance in terms of speed and power, highlighting trade-offs in ALU design.

2.4 Key Challenges Highlighted in Literature

• Designing high-speed ALU with low delay• Reducing power consumption in VLSI systems• Balancing area, speed, and performance• Efficient adder design for arithmetic operations• Limited focus on complete ASIC implementation

2.5 The Project's Contribution in Literature

This project designs a 32-bit ALU using Verilog and implements it using Cadence ASIC flow. It includes both front-end design and backend implementation with functional verification, making it more practical for real-world applications.

CHAPTER-3 OBJECTIVES

3.1 General objectives

The primary objective of this project is to design, simulate, and analyze a **32-bit Arithmetic Logic Unit (ALU)** using Verilog HDL and implement it using Cadence ASIC design flow. The project aims to understand the internal architecture, working, and performance parameters such as power, area, and delay.

- To design and verify a 32-bit ALU capable of performing multiple arithmetic and logical. operations
- To implement the ALU using Verilog HDL and simulate its functionality.
- To analyse performance metrics such as power consumption, delay, and area.

- To understand the complete ASIC design flow using Cadence tools.
- To ensure correct generation of output and status flags (Zero, Carry, Sign).

3.2 Specific Objectives

- To design a 32-bit ALU at RTL level using Verilog• To perform functional verification using a testbench.
- To implement arithmetic and logic operations (ADD, SUB, AND, OR, etc)
- To carry out synthesis and analysis using Cadence tools
- To evaluate performance in terms of power, delay, and area.

3.3 Outcome Statement

This project delivers a functionally verified and synthesizable 32-bit ALU suitable for integration into digital systems. It provides practical experience in ASIC design using Cadence tools and strengthens understanding of arithmetic and logical operations. The design ensures correct functionality and serves as a foundation for future improvements in power and performance.

CHAPTER-4 METHODOLOGY

4.1 Introduction

The methodology for designing the 32-bit ALU focuses on efficient implementation of arithmetic and logical operations using combinational logic. The ALU processes two 32-bit inputs and produces an output based on a control signal.

The design includes arithmetic operations like addition and subtraction using a ripple carry adder, and logical operations such as AND, OR, XOR, and XNOR. All operations are performed in parallel, and a multiplexer is used to select the required output.

The design flow involves RTL coding, simulation, synthesis, and physical design using Cadence tools. The objective is to achieve correct functionality with optimized performance in terms of speed and power.

4.2 Tools and Technology Used

Design Tools:

- Cadence Genus – *Synthesis*
- Cadence Innovus – *Timing Analysis*
- Cadence NC-Verilog / Xcelium – *Simulation*

Hardware Description Language

- **Verilog HDL** ○ Used for RTL design of the 32-bit ALU.
- **System Verilog** ○ Used for writing **testbenches** and verification logic

Operating Environment

- Linux-based Design Environment
- Typically used for running Cadence tools efficiently in an academic or industry setting.

4.3 Design Flow Steps

Operation Selection

In the 32-bit ALU, different operations such as AND, OR, XOR, XNOR, ADD, and SUB are performed in parallel. A multiplexer selects the required output based on the select signal (sel). This ensures fast operation without recalculating results.

Arithmetic Unit (Adder/Subtractor)

Arithmetic operations are implemented using a 32-bit ripple carry adder. Addition is performed using A + B + Cin, while subtraction is implemented using 2's complement method (A + ~B + 1). This allows reuse of the same adder circuit for both operations

Logical Unit

Logical operations such as AND, OR, XOR, XNOR, and NOT are performed using bitwise logic gates. These operations work on all 32 bits simultaneously, ensuring efficient parallel processing.

Status Flag Generation

The ALU generates flags such as Zero, Carry, and Sign based on the result. These flags help in determining conditions like zero output, carry generation, and sign of the result.

ASIC Design Flow

The design follows standard ASIC flow:

- RTL design using Verilog
- Functional simulation
- Synthesis using Cadence Genus
- Physical design using Innovus

CHAPTER 5 DESIGN OF ASYNCHRONOUS FIFO

5.1 Introduction

This chapter explains the design and working of a 32-bit Arithmetic Logic Unit (ALU). The ALU performs arithmetic and logical operations on two 32-bit inputs. The main challenge is to design an efficient ALU that provides correct results with optimized speed and power.

5.2 Steps in Designing

Define Data Width Select 32-bit input size for processing large data.

Implement Arithmetic Unit Use ripple carry adder for addition and 2's complement method for subtraction.

Implement Logical Unit Design AND, OR, XOR, XNOR, and NOT operations using logic gates.

Operation Selection Use a multiplexer controlled by select signal (Sel) to choose output.

Parallel Execution All operations are performed simultaneously for faster output.

Flag Generation Generate Zero, Carry, and Sign flags based on result.

Input Signal Description• A [31:0] First input operand

• **B [31:0]** Second input operand

• **Sel [2:0]** Select signal to choose operation

• **Cin** Carry input for arithmetic operations

5.3 Output Signal Description

result [31:0] Final output of ALU •

Zero-Indicates result is zero

• **Carry-**Carry output from addition

• **Sign-**Indicates sign of result (MSB)

5.4 Summary

This chapter presented the design of a 32-bit ALU, including its operations, inputs, outputs, and working. The ALU performs multiple operations efficiently using parallel processing and multiplexerbased selection, making it suitable for digital system applications.

CHAPTER-6 IMPLEMENTATION

6.1 Verilog code for 32-bit ALU

```
module alu_32bit ( input [31:0] A, B, input
[2:0] sel, input Cin, output reg [31:0] result,
output Zero, Carry, Sign
```

```
);
wire [31:0] and_out, or_out, xor_out, xnor_out, not_a;
wire [31:0] add_out, sub_out; wire c_out_final,
sub_cout; assign and_out = A & B; assign or_out  = A
| B; assign xor_out = A ^ B; assign xnor_out = ~(A ^
B); assign not_a = ~A; // ADD
adder_32bit adder(A, B, Cin, add_out, c_out_final);
// SUBTRACT (2's complement) adder_32bit
subtr(A, ~B, 1'b1, sub_out, sub_cout); always
@(*) begin case (sel)
3'b000: result = and_out;
3'b001: result = or_out;
3'b010: result = xnor_out;
3'b011: result = xor_out;
3'b100: result = add_out;
3'b101: result = sub_out;
3'b110: result = A; 3'b111:
result = not_a; default:
result = 32'b0; endcase
end
assign Zero  = (result == 32'b0);
assign Carry = c_out_final;
assign Sign  = result[31];
endmodule
```

32-bit Ripple Carry Adder module

```
adder_32bit ( input [31:0] A, B,
input Cin, output [31:0] sum,
output cout_final
);
wire [32:0] carry;
assign carry[0] = Cin;
genvar i; generate
```

```
for (i = 0; i < 32; i = i + 1) begin : ripple full_adder
fa(A[i], B[i], carry[i], sum[i], carry[i+1]); end
endgenerate assign cout_final = carry[32];
endmodule
```

1-bit Full Adder module full_adder (

```
input a, b, cin, output sum, cout
);
assign sum  = a ^ b ^ cin; assign cout = (a
& b) | (a & cin) | (b & cin); endmodule
```

6.2 Code Explanation

The 32-bit ALU is designed using Verilog HDL to perform both arithmetic and logical operations. It takes two 32-bit inputs (A and B), a select signal (sel), and a carry input (Cin), and produces a 32-bit result along with status flags.

Logical operations such as AND, OR, XOR, XNOR, and NOT are implemented using bitwise operators. These operations are performed on all 32 bits in parallel, ensuring fast computation.

Arithmetic operations are performed using a 32-bit ripple carry adder. The addition operation computes A + B + Cin, while subtraction is implemented using the two's complement method (A + ~B + 1), allowing reuse of the same adder circuit.

A case statement is used to select the required operation based on the sel signal. This acts as a multiplexer, choosing one output among all computed results.

The ALU also generates status flags. The Zero flag is set when the result is zero, the Carry flag is obtained from the adder output, and the Sign flag is determined from the most significant bit (MSB) of the result.

The ripple carry adder is built using 32 full adders connected in series, where the carry output of one stage is passed to the next stage. Each full adder performs single-bit addition using basic logic equations.

6.3 Test bench for 32-bit ALU

```
module tb_alu_32bit; reg [31:0] A,
B; reg [2:0] sel; reg Cin; wire [31:0]
result; wire Zero, Carry, Sign; //
Instantiate ALU
```

```
        alu_32bit uut (.A(A), .B(B), .sel(sel),Cin(Cin),
        .result(result),
        .Zero(Zero),
        .Carry(Carry)
        ,   .Sign(Sign)
        ); initial begin
        // Initial values
        A = 32'hAAAAAAAA;
        B = 32'h55555555;
Cin = 0;
$display("\n=== PAPER OPERATIONS + EXTENSIONS ===");
// Logical Operations
sel = 3'b000; #10; $display("000 AND:  %h", result);
sel = 3'b001; #10; $display("001 OR:   %h", result);
sel = 3'b010; #10; $display("010 XNOR: %h", result);
sel = 3'b011; #10; $display("011 XOR:  %h", result);
// Arithmetic Operation
sel = 3'b100; #10; $display("100 ADD:  %h C=%b", result, Carry);
// Subtraction
sel = 3'b101; #10; $display("101 SUB:  %h", result);
// Buffer
sel = 3'b110; #10; $display("110 BUF:  %h", result);
// NOT Operation
sel = 3'b111; #10; $display("111 NOT:  %h", result);
$display("\n=== FLAGS TEST ===");
// Zero Flag Test
A = 32'h00000005; B = 32'hFFFFFFFB; sel = 3'b100; Cin = 0;
#10 $display("5 + (-5) = %h Z=%b C=%b S=%b", result, Zero, Carry, Sign);
// Zero case
A = 32'h00000000; B = 32'h00000000; sel = 3'b000; Cin = 0;
#10 $display("ZERO TEST: %h Z=%b", result, Zero);
// Carry test
A = 32'hFFFFFFFF; B = 32'h00000001; sel = 3'b100; Cin = 1;
```

```
#10 $display("CARRY TEST: %h C=%b", result, Carry);
// Overflow test
A = 32'h7FFFFFFF; B = 32'h00000001; sel = 3'b100; Cin = 0;
#10 $display("OVERFLOW: %h C=%b S=%b", result, Carry, Sign);
$display("\n=== ALL TESTS COMPLETE ===");
#50 $finish;
end endmodule
```

6.4 Summary

The 32-bit ALU Verilog design implements a combinational circuit capable of performing multiple arithmetic and logical operations using two input operands and a select signal. Arithmetic operations are carried out using a ripple carry adder, while logical operations are implemented using bitwise operators. A multiplexer selects the final output based on the control signal.

The ALU also generates status flags such as Zero, Carry, and Sign to indicate the result condition. The testbench verifies the functionality by applying different input combinations and checking operations like addition, subtraction, logical functions, and edge cases such as zero, carry, and overflow.

Overall, the design ensures correct operation, efficient implementation, and reliable performance, making it suitable for digital system and ASIC applications.

CHAPTER-7 RESULTS

7.1 Simulation

Simulation of the 32-bit Arithmetic Logic Unit (ALU) involves verifying its functionality for various arithmetic and logical operations using a testbench environment. The simulation is carried out by applying different 32-bit input operands to the ALU and selecting operations through control signals. Initially, a reset condition is applied to ensure all output signals and status flags such as carry, zero, and sign are initialized properly. During simulation, multiple input combinations are provided to the ALU inputs (A and B), and different operation select signals (Sel [2:0]) are applied to perform operations such as addition, subtraction, bitwise AND, OR, and other logical functions. The output result [31:0] is observed for each operation and verified against expected values. Additionally, status flags including carry-out, zero flag, and sign flag are monitored to ensure correct behaviour for each arithmetic and logical computation. output results over time. Each change in the select line produces the appropriate ALU operation,

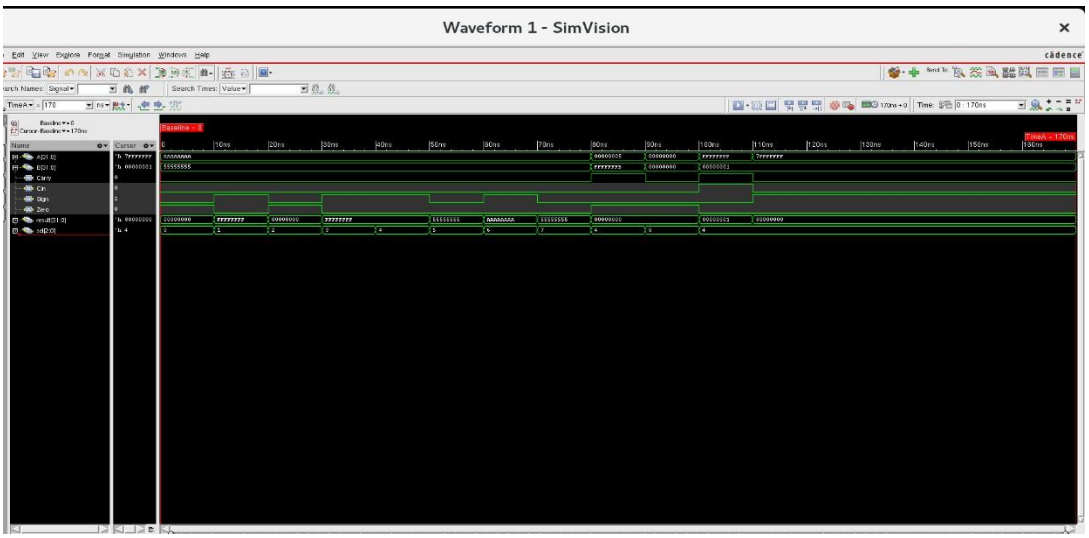

Fig 3: Simulation Waveforms of 32-bit ALU

During simulation of the 32-bit ALU, key events occur based on the changes in input operands and the operation select signals rather than clock-driven write/read operations. The ALU operates combinationally, and the output is updated whenever there is a change in the input signals or control lines. When the reset signal is active, all output signals such as result, carry, zero, and sign flags are initialized to zero to ensure a known starting state. When the reset is inactive, the ALU performs operations based on the select input (Sel [2:0]). For each change in the select line, the corresponding arithmetic or logical operation is executed on the 32-bit input operands A and B. For example, when the select signal corresponds to addition, the ALU computes the sum of A and B and updates the result

along with the carry flag. Similarly, for subtraction, the difference is calculated, and appropriate flags are updated based on the result. For logical operations such as AND, OR, and XOR, the ALU performs bitwise operations on the inputs, and the result is reflected immediately in the output waveform. The zero flag is asserted when the result becomes zero, and the sign flag indicates the most significant bit of the result, representing the sign in signed operations.

The simulation waveform demonstrates that for each change in input or select signal, the ALU responds correctly by producing the expected output without any timing errors or inconsistencies, thereby validating the correct functional behaviour of the 32-bit ALU.

7.2 Synthesis

Define the data width as 32-bit inputs and outputs, along with control signals to select arithmetic and logical operations within the ALU. The design operates on combinational logic without separate clock domains, and all operations are controlled through the select lines.

Implement the RTL design using Verilog, including arithmetic units for addition and subtraction, and logic units for AND, OR, XOR operations. The design also includes status flag generation such as carry, zero, and sign flags to indicate the nature of the result.

Verify functionality through simulation by applying various input combinations and select signals, ensuring correct output results and proper flag behaviour for all supported operations of the ALU.

Apply synthesis constraints such as timing requirements and design optimization goals, and use synthesis tools like Cadence Genus to convert the RTL description into a gate-level netlist using standard cell libraries.

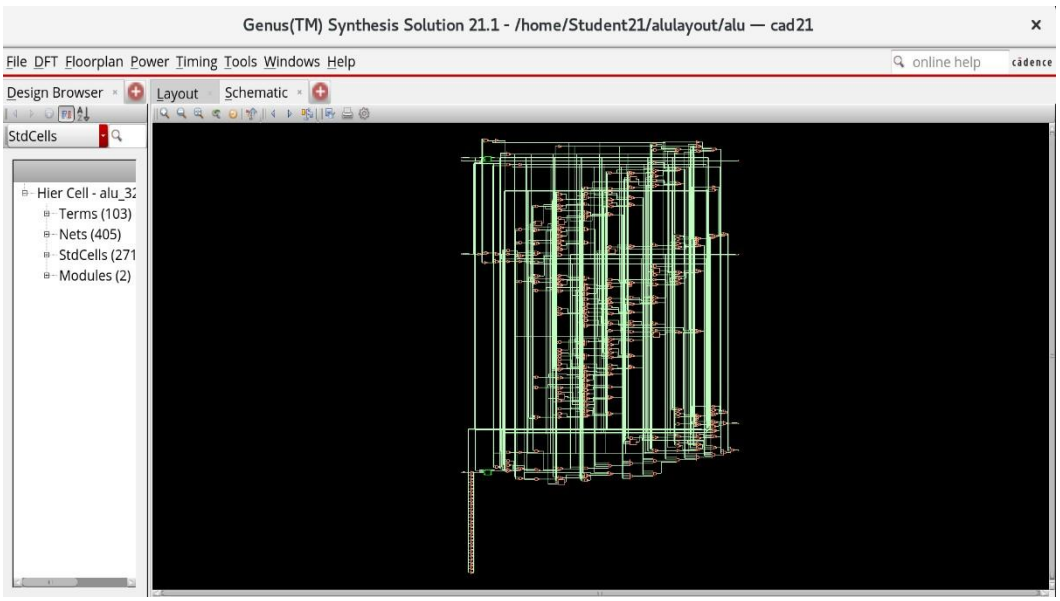

Fig 4: schematic diagram of 32-bit ALU

7.3 Reports

7.3.1 POWER REPORT

- The report shows power consumption breakdown for the 32-bit ALU module synthesized using Cadence Genus.
- Power is divided into leakage, internal, and switching components, with total power being the sum of all three.
- Logic blocks consume the entire power (100%), primarily due to internal operations and switching activity within arithmetic and logic units.
- Other categories such as memory, registers, latches, and pads show zero power consumption, indicating a purely combinational design structure.
- Total power consumed is approximately 88.64 µW, with 12.43% from leakage, 59.60% from internal, and 27.97% from switching.

```
Open ▾                    alu_power.rpt                    Save
                          ~/alulayout/alu

Instance: /alu_32bit
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     1.10201e-05  5.28353e-05  2.47903e-05  8.86456e-05 100.00%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------
    Subtotal     1.10201e-05  5.28353e-05  2.47903e-05  8.86456e-05 100.00%
  Percentage          12.43%       59.60%       27.97%      100.00% 100.00%
  ----------------------------------------------------------------------
```

Fig 5: Power Report of Asynchronous 32-bit ALU

7.3.2 AREA REPORT

```
============================================================

     Instance         Module      Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------------------------
alu_32bit                                338   2619.631     0.000     2619.631 <none> (D)
  adder           adder_32bit             32    629.741     0.000      629.741 <none> (D)
```

Fig 6: Area Report of 32-bit ALU

- The synthesis tool used is Cadence Genus, and the module analyzed is named alu_32bit.
- The design uses a slow technology library under slow operating conditions with an enclosed wire load mode.
- The total number of standard cells used in the design is 338.

The total cell area occupied by the design is 2619.631 units, and net area (routing/interconnect) is reported as zero.

7.3.3 TIMING REPORT

Timing reports generated during **synthesis** provide valuable insights about how well your design meets timing constraints. Here's a breakdown of what to expect and what to look for in the synthesis timing report for an asynchronous FIFO.

```
------------------------------------------------------------------------
|timeDesign Summary
------------------------------------------------------------------------

Setup views included:
  setup
+--------------------------------+-------------+-----------+-----------+
|  Setup mode                    |   all       |  reg2reg  | default   |
+--------------------------------+-------------+-----------+-----------+
|                      WNS (ns):|  -0.152     |  -0.051   | -0.152    |
|                      TNS (ns):|  -0.456     |  -0.112   | -0.456    |
|                 Violating Paths:|    19      |    5      |   14      |
|                       All Paths:|   301      |   145     |  255      |
+--------------------------------+-------------+-----------+-----------+

-------------------------------------------------------------------+
|   DRVs     |                Real        |   Total               |
|------------+-----------------------------+-----------------------+
|            |        Nr nets(terms)|  Worst Vio |  Nr nets(terms)|
|------------+-----------------------------+-----------------------+
| max_cap    |            16 (16) |     0.05  |       16 (16) |
| max_tran   |             0  (0) |        0  |        0  (0) |
| max_fanout |             0  (0) |        0  |        0  (0) |
| max_length |             0  (0) |        0  |        0  (0) |
+------------+-----------------------------+-----------------------+
 Density: 72.28%
 Routing overflow 000% H and 00% V
------------------------------------------------------------------
+ Reported timing to dir timingReports
Total CPU time: 0.37 sec
Total Real time: 6.70 s
MEM Usage: 152.164125 144125 Megatytes
------------------------------------------------------------------
** timeDesign #1 (finish): cpu/real = 000:00.4/000.00:6.8 (0.0), total sesion Pupal = 152.1M (0.0)
innovus 1>
```

Fig 7: Timing Report of 32-bit ALU

- **Stage**: The report is from the Pre-route phase — placement is completed while routing is yet to be finalized.
- **WNS (Worst Negative Slack)** is slightly negative, indicating minor setup timing violations in the ALU design.
- **TNS (Total Negative Slack)** shows small accumulated delay across paths, reflecting limited timing issues.
- A few setup paths are violating timing constraints out of the total analyzed paths in the design.
- **Max capacitance violation** violations are observed on a few nets, which can be improved using buffering techniques.
- No violations are observed for **maximum transition, fanout, or net length** - indicating clean signal conditions.
- **Core density is 70-72%**, representing efficient utilization of available layout area.
- **Routing overflow is 0%**, indicating no congestion and proper placement of standard cells.

- Overall, the design shows minor **timing issues** and small DRVs, which can be resolved through optimization before final implementation.

7.4 Physical Design

The physical design process is the stage in VLSI design where a synthesized netlist is transformed into a physical layout that can be fabricated on a silicon chip. It starts with **floorplanning**, where the chip area is defined, and major functional blocks are placed to optimize performance and area usage. This is followed by **placement**, where standard cells are positioned within the designated blocks while minimizing wire length and congestion. Next is **clock tree synthesis (CTS)**, which ensures that the clock signal reaches all sequential elements with minimal skew and delay. After that, **routing** is performed to connect the placed cells using metal layers, ensuring signal integrity and meeting design rules. Once routing is complete, various **design rule checks (DRC)** and **layout versus schematic (LVS)** checks are carried out to verify the physical layout against manufacturing constraints and logical connectivity. The process concludes with **parasitic extraction** and **static timing analysis (STA)** to confirm that the design meets all timing requirements. If successful, the design is then ready for **tape-out** and fabrication.

FLOORPLAN

- Floorplan can be done by using cadence innovus tool
- Initialize the design using by loading the netlist, LEF files, and SDC constraints.
- Define the floorplan with floorplan, specifying core utilization, aspect ratio, and core margins.

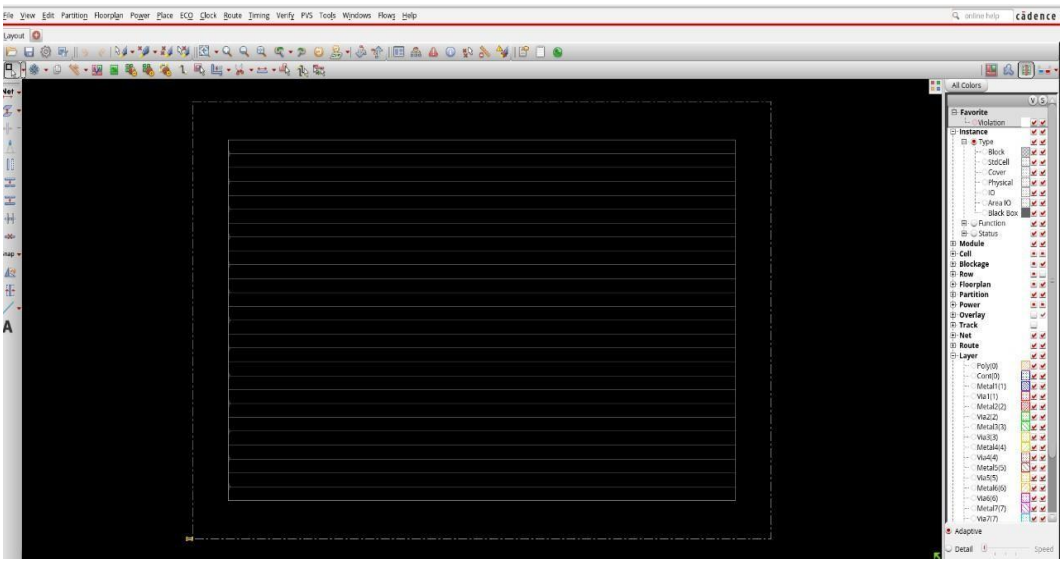

Fig 8: Floorplan of 32bit ALU

○ POWERPLAN

The power planning for the 32-bit Arithmetic Logic Unit (ALU) was carefully implemented to ensure robust and reliable power delivery across the entire layout. A single power domain was used, with global power (VDD) and ground (VSS) nets distributed throughout the design. Power rings were created around the core area using higher metal layers such as Metal9 for horizontal segments and Metal8 for vertical segments, providing a strong foundation for power distribution. To support internal power delivery, a uniform grid of power straps was introduced within the core, utilizing metal layers like Metal7, Metal8, and Metal9. These straps were placed at regular intervals to maintain uniform power distribution and reduce voltage drop across the design. Tap cells were inserted between standard cell rows to connect the substrate to the power supply and prevent latch-up conditions. The power grid was connected to all standard cells through multiple vias to ensure efficient current flow and minimize IR drop. The overall core utilization was maintained around 70–75%, allowing sufficient space for routing and power distribution. No DRC violations or power integrity issues such as shorts or opens were observed, indicating a clean and stable power network. This power plan ensures reliable operation of the 32-bit ALU under various operating conditions and supports consistent performance across all functional scenarios.

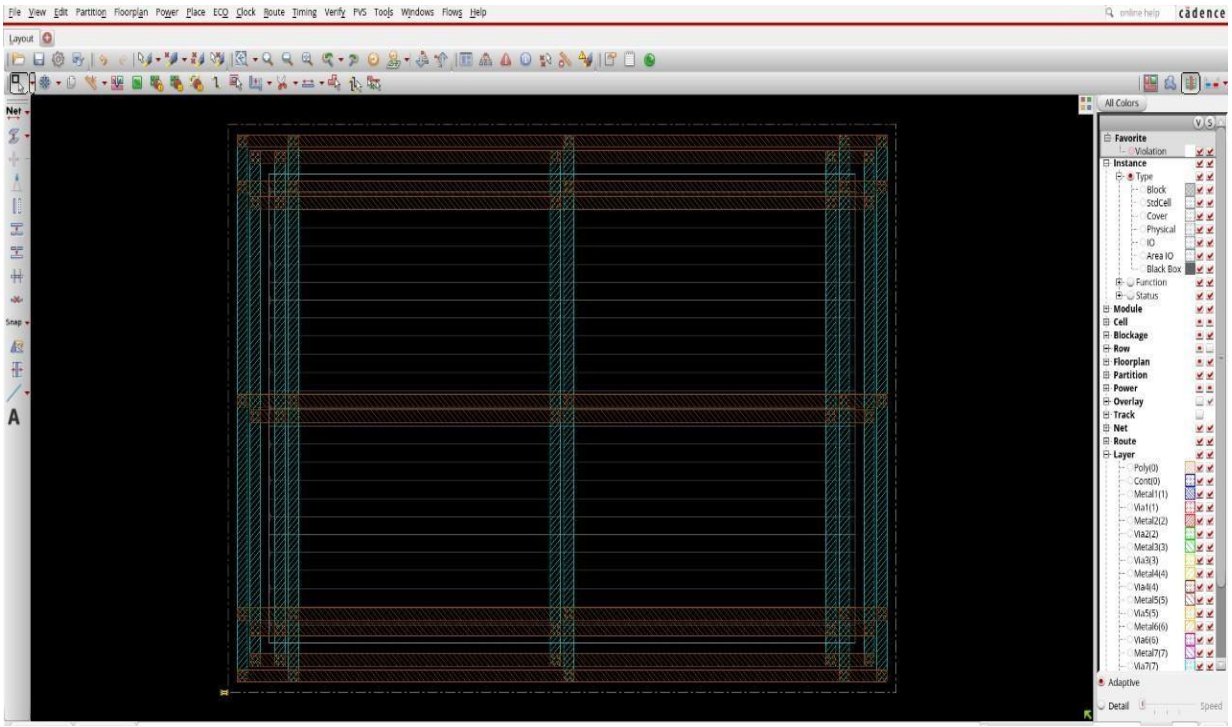

Fig 9: Power plan of 32bit ALU

Placement and Route

- Physically organize arithmetic and logical functional units within the ALU to minimize interconnection complexity and improve overall performance.
- Place critical components such as adders and multiplexers closer to each other to reduce routing delay and ensure faster signal propagation.
- Group related logic blocks like arithmetic units, logic units, and flag generation circuits tightly to optimize area and connectivity.
- Keep high-speed signal paths such as carry propagation and result outputs short and directly routed to minimize delay and signal degradation.
- Apply placement constraints or floor planning regions to guide the tool for efficient layout organization and prevent unnecessary routing congestion.

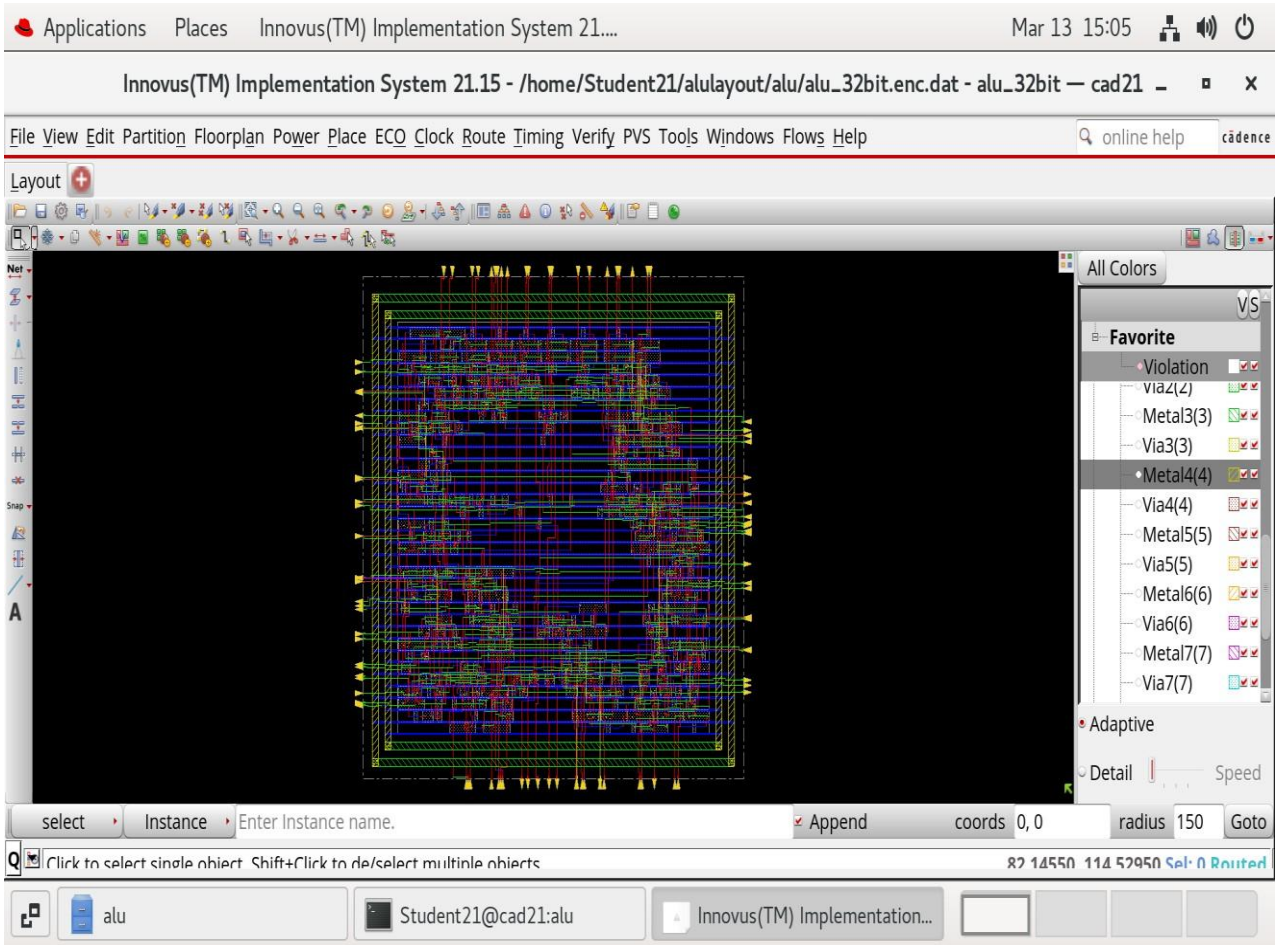

Fig 10: Placement of 32bit ALU

- Ensure a balanced and efficient clock distribution network is implemented during clock tree synthesis to minimize skew and delay across all sequential elements.
- Avoid routing critical paths through congested or high-noise regions to maintain signal integrity and ensure reliable operation of the ALU.
- For FPGA implementations, utilize dedicated logic resources and place ALU functional units close to each other to optimize performance and reduce routing complexity.
- Use proper buffering and fanout control for clock and control signals to maintain timing accuracy and prevent excessive delay in signal propagation.
- After routing, perform comprehensive static timing analysis to verify that all timing constraints are met and no setup or hold violations are present.

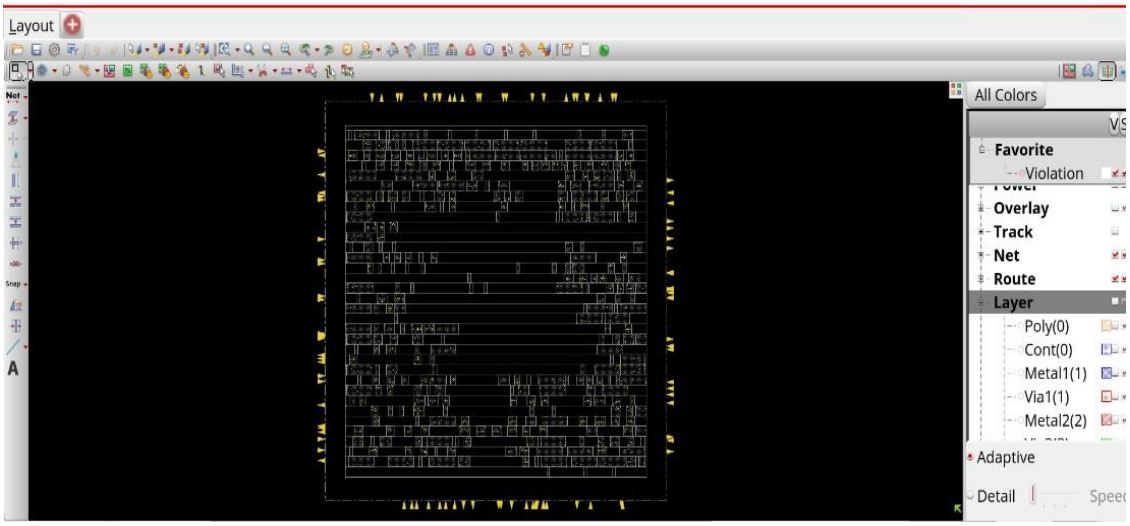

Fig 11: Placement & Routing of 32bit ALU

Clock Tree Synthesis

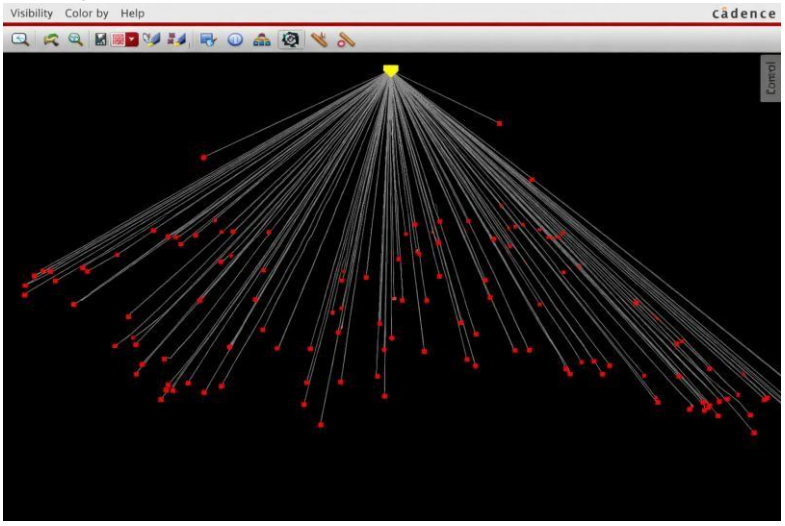

Fig 12: Clock Tree Synthesis of 32bit alu

The figure shows the Clock Tree Debugger in Cadence Innovus, specifically for the Clock Tree Synthesis (CTS) of the 32-bit Arithmetic Logic Unit (ALU) design. In this view, the clock tree appears as a branching structure where the clock signal originates from a source (such as a clock port) and is distributed through a network of buffers and interconnects to multiple endpoints. These endpoints, known as clock sinks, are typically the clock input pins of flip-flops used in the design. Each red square in the structure represents a clock sink that must receive the clock signal with proper timing and synchronization.

A key objective of CTS is to achieve low clock skew across the design. Clock skew refers to the variation in arrival times of the clock signal at different sinks. Minimizing skew is essential to ensure that all sequential elements operate correctly and meet setup and hold time requirements. In the 32-bit ALU design, a single clock domain is used, and the clock

tree is optimized to provide uniform clock distribution across all sequential components. The CTS process ensures that buffers are inserted appropriately and routing paths are balanced so that the clock signal reaches all parts of the ALU with minimal delay differences. This balanced clock distribution improves timing reliability and overall performance of the design. The resulting clock tree structure confirms that the ALU receives a stable and synchronized clock signal, enabling accurate and efficient operation under all conditions.

○ Optimization

After the initial clock tree synthesis (CTS), optimization is a crucial step to ensure that the clock network meets all timing, power, and area constraints. In the case of a 32-bit Arithmetic Logic Unit (ALU), where a single clock domain is used, clock tree optimization is performed to achieve balanced clock distribution and minimize skew across all sequential elements. The optimization process focuses on improving timing performance by adjusting buffer placement, reducing insertion delay, and eliminating any potential timing violations while maintaining overall design efficiency.

- Minimize clock skew to ensure all sequential elements receive the clock signal at nearly the same time across the ALU design.
- Insert and resize buffers to effectively drive large fanouts and maintain strong and stable clock signal distribution throughout the circuit.
- Balance clock latency across all clock sinks to improve timing predictability and reduce the chances of setup and timing violations.
- Add delay buffers on shorter paths to correct hold time violations and ensure proper synchronization between all sequential elements.
- Optimize power by reducing unnecessary clock activity and applying clock gating techniques where possible to improve overall efficiency.
- Reroute congested clock paths to avoid interference with other signals and ensure clean and reliable routing throughout the ALU design.
- Maintain a well-structured clock distribution network to ensure proper signal flow and balanced timing across all parts of the circuit.
- Ensure critical sequential elements are placed appropriately to support reliable operation and maintain proper timing behavior across the design. **○ Optimized Power Report**

Power Details Report — cad21

Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)
Generated on: Feb 25 2026 15:19:36
Module: design:alu_32bit
Technology library: slow
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed

Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
alu_32b...[23].fa	1	84.476	505.114	111.780	616.894
alu_32b...[24].fa	1	84.476	505.114	111.780	616.894
alu_32b...[25].fa	1	84.476	505.114	111.780	616.894
alu_32b...[26].fa	1	84.476	505.114	111.780	616.894
alu_32b...[27].fa	1	84.476	505.114	111.780	616.894
alu_32b...[28].fa	1	84.476	504.688	113.805	618.493
alu_32b...[29].fa	1	84.476	505.114	111.780	616.894
alu_32b...[30].fa	1	84.476	507.538	85.860	593.398
alu_32b...[31].fa	3	88.570	309.517	87.379	396.896
alu_32bit	338	11020.057	52835.251	24790.292	77625.543

Fig 13: optimized Power Report

Total power consumption of the 32-bit ALU is approximately 8.86456×10^{-5} W.

Internal power is 5.28353×10^{-5} W, contributing about 59.60% of total power.

Switching power is 2.47903×10^{-5} W, contributing about 27.97% of total power.

Leakage power is 1.10201×10^{-5} W, contributing about 12.43% of total power.

The entire power consumption is due to logic components (100%), as the ALU is mainly combinational. Combinational logic consumes 0.2003 W (11.65%).

No power is consumed by memory, registers, latches, or clock groups in the design.

The power distribution indicates that most of the power is consumed by arithmetic and logical operations.

The design operates under a single power domain with uniform voltage supply across all components.

The contribution of each functional unit (such as full adders) is similar, indicating balanced design behaviorTotal design capacitance is 4.12367 femtofarads.

The design consists of 338 standard cells, contributing to overall power consumption.

○ Optimized Timing Report

```
Setup views included:
~setup
+-------------------------------------------------------------------+
|                          |   all    | reg2reg    |  default    |
+-------------------------+----------+------------+-------------+
|                WNS (ns)  | ≈ 0.004  |   0.000    |  ≈ 0.0044   |
|                TNS (ns)  |  0.000   |   0.000    |   0.0000    |
|        Vilolating Paths  |     0    |     0      |      0      |
|              ALL Pathss  |   484    |   317      |      1      |
+-------------------------+----------+------------+-------------+

+ DRVs                    |        Real        |       Total        |
|-------------------------------------------------------------------|
|          | Nr nets(terms) | Worst Viol |   | Nr nets(terms)   |  |
+----------+----------------+------------+---+------------------+--+
| max_cap  |       0 (0)    |     0.00   |   |        0 (0)     |  |
| max_tran |       0 (0)    |     0.00   |   |        0 (0)     |  |
| max_fanout       0 (0)    |     0.00   |   |        0 (0)     |  |
| max_length       0 (0)    |     0.00   |   |        0 (0)     |  |
+-------------------------------------------------------------------+

Density: 75.12%
Routing Overflow  0% H and 000% V
```

Fig 14: Optimized Timing Report for Setup

The image shows a time Design Summary report for setup analysis of the 32-bit ALU. The analysis includes three views: all, reg2reg, and default. The Worst Negative Slack (WNS) is approximately 0.004 ns for the all view and close to 0.000 ns for reg2reg paths, indicating that all critical paths meet timing requirements after optimization. The Total Negative Slack (TNS) is 0.000 ns, and there are no violating paths in any view (Violating Paths: 0), confirming that the design has achieved complete timing closure. A total of 484 timing paths were analyzed, all of which satisfy setup timing constraints. The report also indicates a routing density of approximately 75.12%, which reflects efficient utilization of the design area. There is no routing overflow in either horizontal or vertical directions, indicating that the placement and routing are congestion-free. Additionally, no design rule violations (DRVs) such as maximum capacitance, transition, or fanout are observed, confirming a clean and optimized design. The timing analysis process is completed efficiently with minimal computational resources.

Overall, the 32-bit ALU design is fully optimized, timing clean, and suitable for final implementation.

○ GDS File

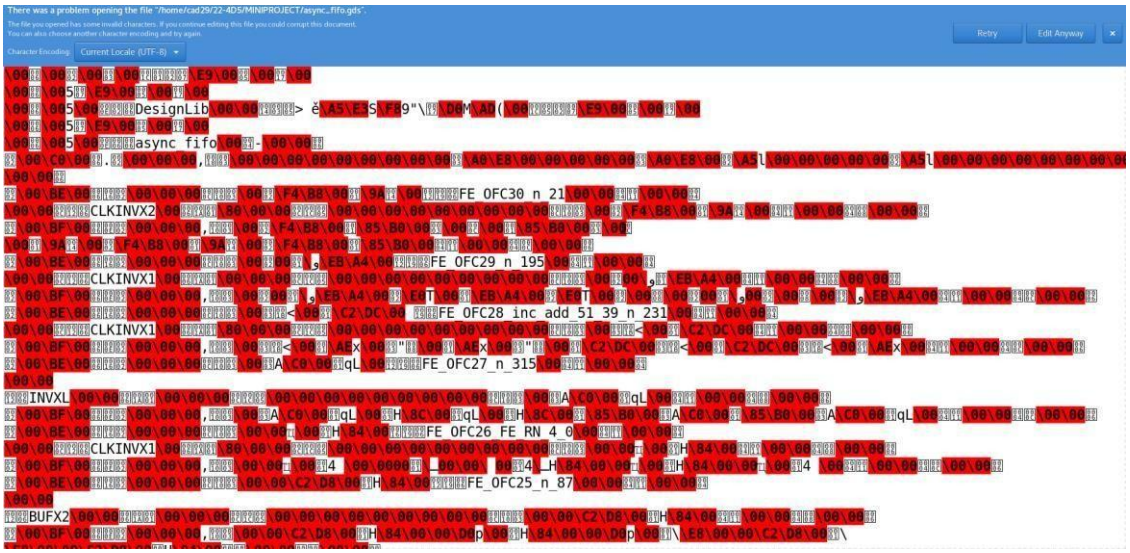

Fig 15: GDS File of 32bit ALU

A GDS file (Graphic Data System II), commonly referred to as GDSII, is a standard binary file format used in VLSI design to represent the physical layout of a 32-bit Arithmetic Logic Unit (ALU). It contains all the geometric shapes, layers, and structures required for chip fabrication. Generated by EDA tools like Cadence Innovus, Synopsys IC Compiler, or KLayout, the GDS file serves as the final blueprint of the ALU design, defining metal layers, vias, diffusion regions, and other key elements across multiple layers.

This file uses a hierarchical structure to manage repeated design blocks efficiently and supports extremely precise dimensions, essential for modern nanometer-scale technologies. The GDS file is typically encrypted to protect intellectual property, ensuring the design cannot be reverse-engineered or accessed by unauthorized users. As such, it is treated as a highly sensitive asset in the ASIC design flow.

Once the design passes verification checks like DRC (Design Rule Check) and LVS (Layout Versus Schematic), the GDS file is sent to the semiconductor foundry for mask generation and fabrication. Despite newer formats like OASIS emerging, GDSII remains the industry standard due to its wide tool support and reliability. It marks the critical transition from the digital layout of the 32-bit ALU to physical silicon.

CHAPTER8 CONCLUSION

The ASIC design and verification of a 32-bit Arithmetic Logic Unit (ALU) is a fundamental and significant project in the field of VLSI and digital system design. In modern System-on-Chip (SoC) architectures, the ALU plays a critical role as the core computational unit responsible for executing arithmetic and logical operations. Its performance directly impacts the overall speed, efficiency, and reliability of digital systems.

This project demonstrates the design of a 32-bit ALU capable of performing multiple operations such as addition, subtraction, and logical functions including AND, OR, XOR, XNOR, and NOT. The use of a ripple carry adder for arithmetic operations and two's complement method for subtraction ensures an efficient and reusable hardware structure. The generation of status flags such as Zero, Carry, and Sign further enhances the ALU's capability to support processor-level decision-making.

From a design perspective, the project emphasizes achieving a balance between power, area, and timing while maintaining correct functionality. The implementation using Verilog HDL, followed by functional verification through a comprehensive testbench, ensures that the ALU operates correctly under various input conditions, including edge cases. The complete ASIC design flow using Cadence tools, including synthesis, placement, routing, and analysis, provides practical exposure to real-world chip design methodologies.

The applications of a 32-bit ALU are extensive, as it forms a core component in microprocessors, digital signal processors (DSPs), embedded systems, and various high-performance computing devices. Its modular and scalable architecture makes it suitable for integration into complex digital systems.

From an academic and learning perspective, this project provides strong foundational knowledge in digital design, hardware description languages, and ASIC implementation. It also opens opportunities for further enhancements such as improving speed using advanced adder architectures, reducing power consumption through optimization techniques, and extending functionality for more complex operations.

In conclusion, the designed 32-bit ALU achieves functional correctness, efficient implementation, and practical relevance, making it a valuable building block for modern digital and VLSI systems

8.1 ADVANTAGES

The design and implementation of a 32-bit Arithmetic Logic Unit (ALU) as an ASIC provide several advantages in terms of performance, power efficiency, and reliability. As a core computational unit in digital systems, the ALU is responsible for executing arithmetic and logical operations efficiently.

- **High Performance –** ASIC-based implementation enables faster computation with reduced propagation delay, improving overall system speed.
- **Low Power Consumption –** Optimized combinational logic and efficient design techniques minimize switching activity and reduce power usage.
- **Efficient Area Utilization** – Custom-designed ALU architecture ensures compact silicon area compared to general-purpose implementations**.**
- **Parallel Operation –** Arithmetic and logical operations are executed simultaneously, resulting in faster output generation**.**
- **Scalable Design –** The 32-bit architecture can be extended to higher bit-widths (e.g., 64-bit) with minimal structural changes.
- **Accurate Flag Generation** – Status flags such as Zero, Carry, and Sign provide essential information for processor decision-making**.**

8.2 DISADVANTAGES

While the ASIC design and implementation of a 32-bit Arithmetic Logic Unit (ALU) provide significant benefits in terms of performance and efficiency, it also involves several challenges and limitations. Designing a high-performance ALU requires careful optimization of speed, power, and area, along with rigorous verification to ensure correct functionality under all operating conditions.

- **High Development Cost –** ASIC design, verification, and fabrication involve significant cost compared to FPGA-based implementations.
- **Long Design Cycle –** The complete ASIC flow, including RTL design, simulation, synthesis, physical design, and verification, requires considerable time.
- **Difficult to Modify After Fabrication –** Once the ALU is fabricated, any design errors or required improvements need a costly redesign and re-fabrication.
- **Design Complexity** – Implementing a 32-bit ALU with multiple operations and flag generation increases design complexity and requires careful planning.
- **Timing Optimization Challenges –** Ensuring that the ALU meets timing constraints (especially in ripple carry adders) can be difficult due to propagation delays.

8.3 REALTIME APLICATIONS

A 32-bit Arithmetic Logic Unit (ALU) is a fundamental component in digital systems and is widely used in real-time applications where fast and efficient computation is required. It performs arithmetic and logical operations that are essential for data processing, decisionmaking, and control operations in various systems. The implementation of a 32-bit ALU in ASIC design ensures high performance, low power consumption, and reliable operation, making it suitable for a wide range of real-world applications.

- **Microprocessors and CPUs** o The ALU acts as the core computational unit, executing arithmetic and logical instructions required for program execution.
- **Digital Signal Processing (DSP) Systems** o Used in applications such as audio processing, filtering, and signal analysis where fast mathematical operations are required.
- **Embedded Systems**
  - o Plays a key role in microcontrollers used in devices like home appliances, industrial controllers, and IoT devices.
- **Image and Video Processing Systems** o Performs pixel-level operations such as addition, subtraction, and logical transformations in real-time processing.
- **Communication Systems**
  - o Used in encoding, decoding, and data manipulation processes in wireless and wired communication systems.

- **Graphics Processing Units (GPUs)** o Performs large-scale parallel arithmetic operations required for rendering images and graphics.
  - o Used in control systems such as engine control units (ECUs), driver assistance systems, and sensor data processing.
- **Scientific and High-Performance Computing** o Supports complex mathematical computations in simulations, modeling, and data analysis.

CHAPTER9

FUTURE SCOPE

The ASIC design and implementation of a 32-bit Arithmetic Logic Unit (ALU) is a foundational project in the field of VLSI and digital system design. As the core computational unit in processors, the ALU plays a critical role in executing arithmetic and logical operations. With continuous advancements in semiconductor technology, increasing demand for high-performance computing, and the evolution of modern architectures, the scope for improving and extending ALU designs continues to grow. The following points highlight the future scope of this project:

- The 32-bit ALU can be integrated into complex System-on-Chip (SoC) designs where multiple processing units require efficient computation.
- The design can be scaled to 64-bit or higher ALUs to support modern high-performance processors.
- Replacing ripple carry adders with faster adders like Carry Look-Ahead (CLA) or Carry Select Adders can significantly reduce delay.
- Implementation of low-power techniques such as clock gating, operand isolation, and voltage scaling to reduce power consumption.
- ALUs can be optimized for specialized operations required in machine learning, signal processing, and data-intensive applications.
- Introducing pipelining and parallel processing to improve throughput and performance in highspeed systems.
- Creating standardized and reusable ALU modules for integration into different processor architectures and open-source platforms like RISC-V.
- Adoption of advanced verification methodologies such as UVM and formal verification for improved reliability.
- Designing ALUs with built-in security features to prevent data manipulation and ensure secure computations.

- Use of ALU designs in emerging domains such as edge computing, IoT devices, and real-time embedded systems.
- The ALU can be adapted for use in emerging technologies such as quantum-classical hybrid systems and neuromorphic computing, where efficient arithmetic and logical processing is required.
- The 32-bit ALU remains highly relevant in academic projects, providing a strong foundation for understanding digital design, VLSI concepts, and processor architecture.

REFERENCES

[1] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 5th Edition, Pearson, 2013.

[2] Neil H. E. Weste and David Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th Edition, Pearson, 2011.

[3] Jan M. Rabaey, Anantha Chandrakasan, and Borivoje Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd Edition, Pearson, 2003.

[4] Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2009.

[5] Sung-Mo Kang and Yusuf Leblebici, *CMOS Digital Integrated Circuits: Analysis and Design*, 3rd Edition, McGraw-Hill, 2003.

[6] R. P. Jain, *Modern Digital Electronics*, 4th Edition, McGraw-Hill Education, 2009.

[7] Clifford E. Cummings, "Simulation and Synthesis Techniques for Digital Design," SNUG Conference Papers, 2002.

[8] Cadence Design Systems, *Genus Synthesis Solution User Guide*, 2023.

[9] Cadence Design Systems, *Innovus Implementation System User Guide*, 2023.

[10] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson, 2003